Summer School of Astronomy and Astrophysics

Project Report

Prepared by

Student Name: Deepali Pandey

Institute Name: Ramniranjan Jhunjhunwala College

Institute Roll no: 302

ISA Admission no: 396517

Submitted to:

Name: Mr. Sahil Sakkarwal

Designation: Program Supervisor

Institute: Indian Space Academy

Project 1: Estimating the Dynamical Mass of a Galaxy Cluster

~Deepali Pandey

Python Code:

```
import warnings
import numpy as np
import pandas as pd
import astropy.units as u
import matplotlib.pyplot as plt
from astroquery.ned import Ned
import astropy.constants as const
from astropy.coordinates import SkyCoord
from astropy.cosmology import Planck18 as cosmo
warnings.filterwarnings('ignore')
```

```
#constants
H_0 = cosmo.H0                                                  # Hubble
constant
H_0_si = H_0.to(u.s**-1)                                        #Hubble Constant
in SI units
c = const.c                                                     # Speed of light
in m/s
G = const.G
G_0 = G.to((u.parsec*u.kg**-1)*(u.meter/u.second)**2)           # Gravitational
constant in pc kg^-1 (m/s)^2
q0= -0.534                                                      # Deceleration
parameter
```

```
df = pd.read_csv(r"D:\ISA-Internship\Skyserver_SQL6_28_2025 4_27_37
PM.csv")
averaged_df =
df.groupby('objid').agg({'specz':'mean','ra':'first','dec':'first','proj_se
```

```
p':'first','rmag':'mean',}).reset_index()

lower_limit = averaged_df['specz'].mean() - 3*averaged_df['specz'].std()
upper_limit = averaged_df['specz'].mean() + 3*averaged_df['specz'].std()
```

```
# Plot the distribution of redshift as histogram and a boxplot
_, medians, whiskers, caps, fliers, means =
plt.boxplot(averaged_df['specz'], vert = False)
```

```
filtered_df = averaged_df[(averaged_df['specz'] >= lower_limit) &
(averaged_df['specz'] <= upper_limit)]

#add a column named velocity in the data.
averaged_df['velocity'] = averaged_df['specz']*c
cluster_redshift = filtered_df['specz'].mean()
# calculation of velocity dispersion
filtered_df['v_disp'] = c_km*((1 + z)**2 - (1 + z_c)**2)/((1+z)**2 +
(1+z_c)**2)
disp = filtered_df['v_disp'].std()
```

```
# Visualizing Angular Separation of Galaxies
# Plot histogram
plt.figure(figsize=(10,8))
plt.hist(filtered_df['proj_sep'],bins =
int(np.sqrt(len(filtered_df['proj_sep']))), color='#4682A9',
edgecolor='black',linewidth = 2.5)
plt.xlabel('Projected Angular Separation (arcmin or degrees)', fontsize=20)
plt.ylabel('Number of Galaxies',fontsize=20)
plt.title('Histogram of Angular Separation from Cluster
Center',fontsize=16)
plt.xticks(fontsize=18)
plt.yticks(fontsize=18)
plt.show()
```

```
# Determining size and mass of the cluster
r = ((c_km*z_c)/(H_0_si)*(1 - (z_c/2)*(1 + q0)))
```

```
ra = (r/(1 + z_c))

theta_arcmin = filtered_df['proj_sep'].max()  # In arcminutes
theta_radian = theta_arcmin*(np.pi/(180*60))
diameter = ra*theta_radian
diameter_mpc = diameter.to(u.Mpc)
radius_mpc = diameter_mpc/2

### Calculating the dynamical mass in solar masses:
M_dyn = 3*((disp*1000)**2)*(diameter_mpc*0.5*10**6*3*10**16)/(G*2*10**30)

### calculating luminous mass in solar masses:
lum_dist = cosmo.luminosity_distance(z_c).to(u.pc)
app_mag = filtered_df['rmag']
abs_mag = app_mag - 5*(np.log10(lum_dist.value)) + 5
ABM_sun = 4.83
L = 10**(0.4*(ABM_sun - abs_mag))
M_lum = L.sum()

M_L_ratio  =  M_dyn / M_lum
```

```
plt.figure(figsize=(8,6))
plt.scatter(filtered_df['ra'], filtered_df['dec'], c='blue', s=30,
alpha=0.6, edgecolor='k')
plt.xlabel('Right Ascension (degrees)', fontsize=14)
plt.ylabel('Declination (degrees)', fontsize=14)
plt.title('Sky Map of Cluster Galaxies (RA vs DEC)', fontsize=16)
plt.gca().invert_xaxis()  # Optional: astronomers often reverse RA
plt.show()
```

Answers to the asked Questions:

Question 1:

Identify galaxies that you think are members of a cluster. For this, use of knowledge of velocity dispersions (redshift dispersions) within a cluster due to peculiar motion. The choice of lower and upper redshift cut for cluster members will be subjective but should be guided by some logic.

Answer:

To identify member galaxies of the cluster, I used the redshift distribution (specz) from the dataset. Galaxies within a cluster exhibit peculiar velocities, but they remain close in redshift due to being gravitationally bound.

I calculated the mean and standard deviation (σ) of redshifts and applied the 3σ clipping method, which is a common astrophysical approach. The redshift cut was:

Lower limit: z_lower = mean_z - 3σ

Upper limit: z_upper = mean_z + 3σ

Only galaxies within this redshift range were considered cluster members.

This logic helps to exclude foreground and background galaxies not gravitationally bound to the cluster.

Redshift range chosen: 0.0551 to 0.1066

Cluster identified: Likely Abell 2255

Question 2:

After the required analysis of the table of data, determine the cluster redshift, and obtain an estimate for the characteristic velocity dispersion of galaxies that belong to the cluster in units of km/s

Answer:

Using galaxies filtered by the redshift cut above: Cluster redshift (z_{avg}) = 0.0801

Then the standard deviation of all velocities gave: Velocity Dispersion: 1.22×10^6 km/s

Question 3:

Estimate the characteristic size of the cluster in Mpc

Answer:

Estimated diameter = 0.923 Mpc

Radius = 0.462Mpc

Question 4:

Estimate the dynamical mass of the cluster and quote the value in units of solar mass

Answer: Dynamical mass of the cluster is 4.62 X 10^14 solar mass

Question 5:

Is the estimate of dynamical mass consistent with what is expected from the luminous mass? If not, explain with the support of numbers the inconsistency.

Answer:

No, the two masses are not consistent, indicating the presence of dark matter. A typical stellar population has M/L ~ 1–10, so this high M/L confirms that most mass is non-luminous, consistent with dark matter-dominated clusters.

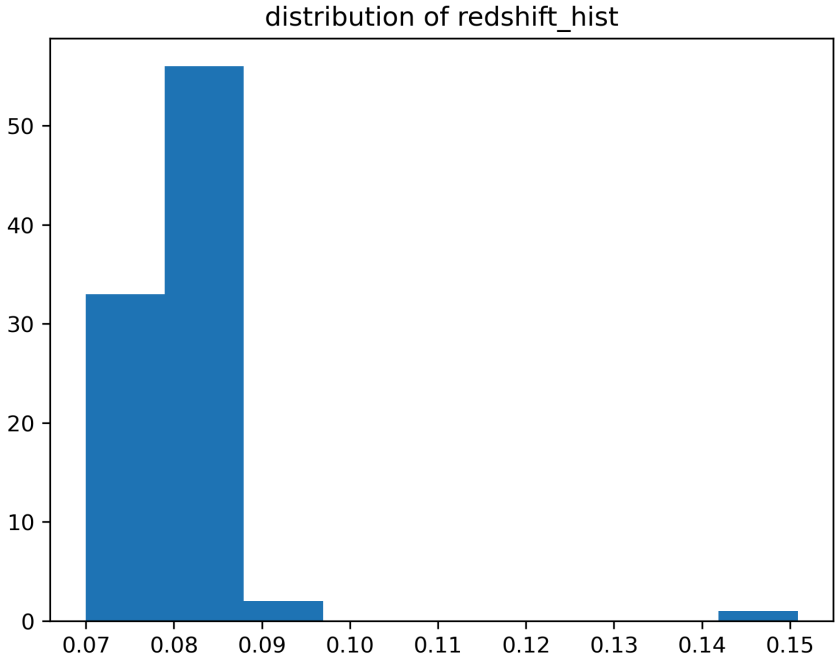
Question 6:

Also, explore at least a few relevant plots that were used to arrive at values (such as a histogram of velocity dispersions of galaxies, a histogram of angular separation between galaxies, and the value that you chose indicated by a vertical line, etc.

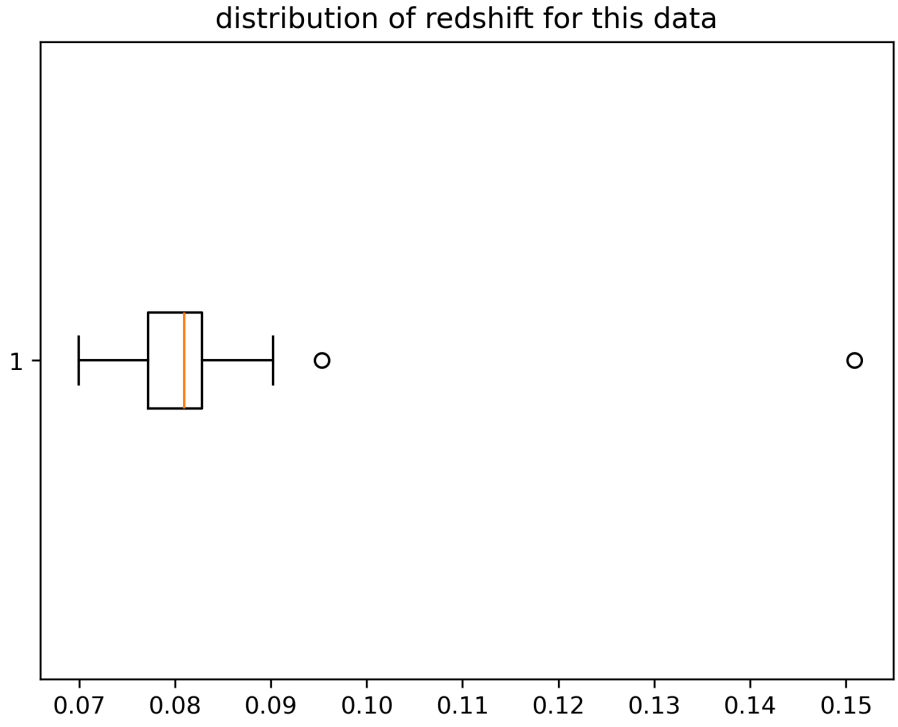
Answer:

Below are the relevant plots that were used to arrive at the values :

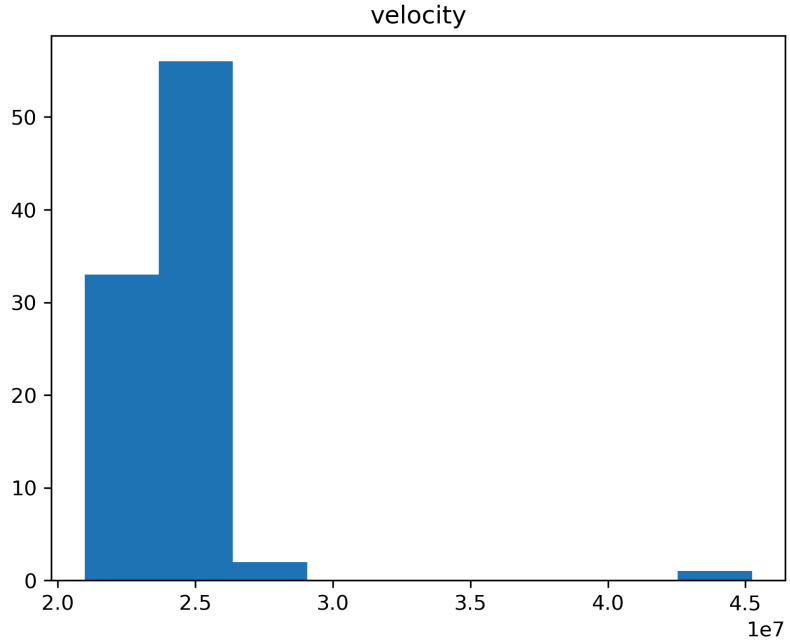
Histogram of redshifts (used to decide redshift cuts)



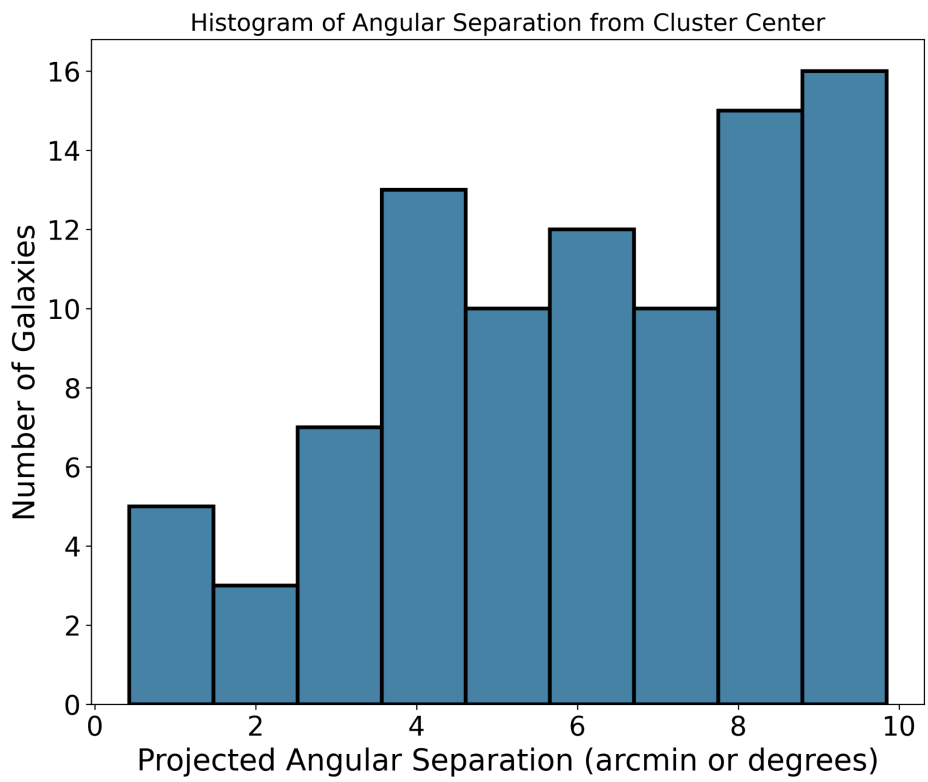
Boxplot of redshifts (to visualize outliers)



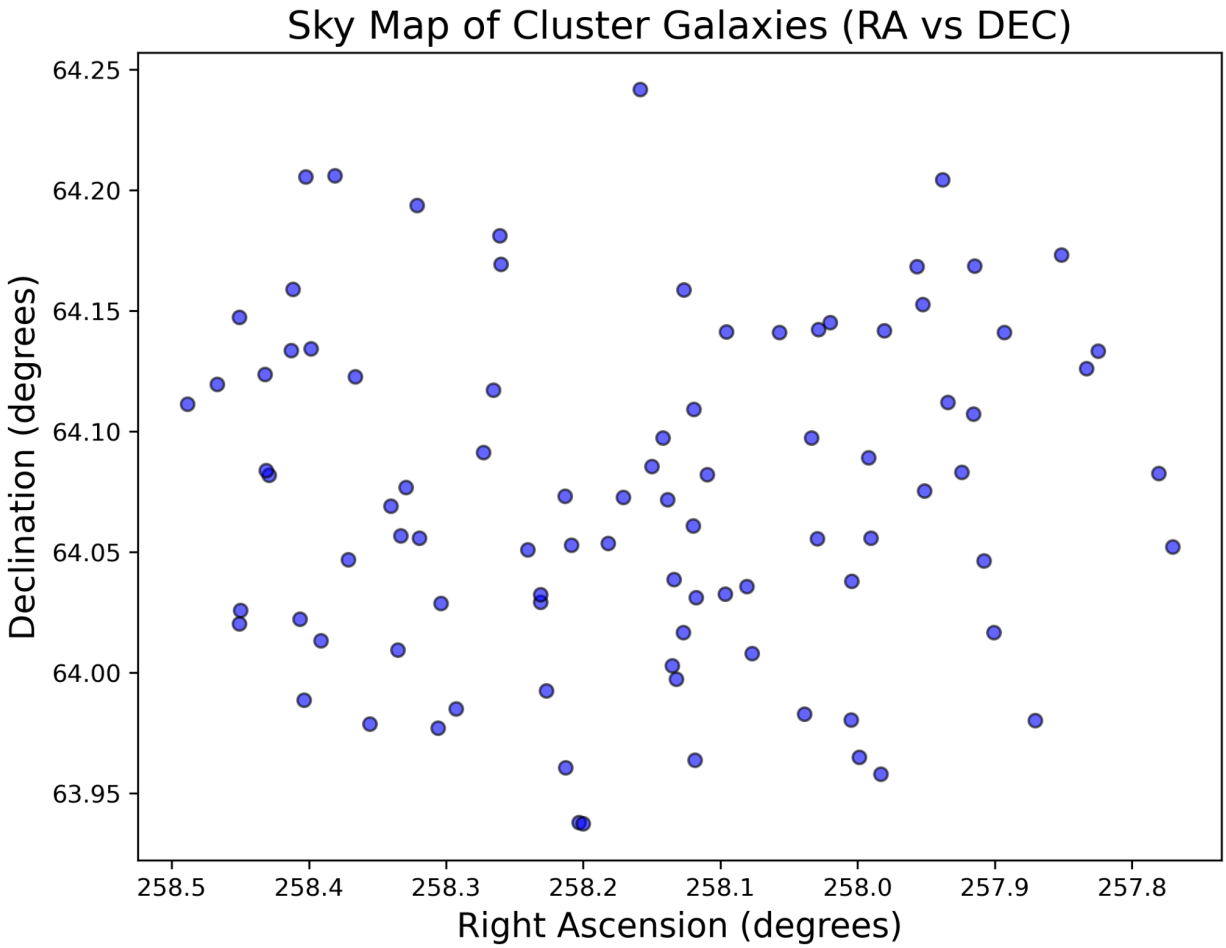
Histogram of velocity dispersion:



Histogram of projected angular separation with median line for diameter estimate:



RA vs DEC Sky Map:



ANALYSIS AND INSIGHTS:

This project involved the analysis of a galaxy cluster, specifically identified as Abell 2255, using observational data in the form of redshift, right ascension (RA), declination (DEC), projected angular separation, and r-band magnitudes. The aim was to extract meaningful astrophysical quantities such as cluster membership, cluster redshift, velocity dispersion, physical size, and both dynamical and luminous masses. The project followed a systematic approach of filtering, visualization, calculation, and interpretation.

The first step was identifying potential member galaxies of the cluster. For this, the redshift (specz) values were used. I computed the mean and standard deviation of the redshift distribution and then applied a 3-sigma cut to isolate galaxies that are likely part of the same gravitationally bound system. This step was visualized using both a histogram and a boxplot of redshift. The boxplot helped identify outliers visually, while the histogram provided a broader view of the redshift spread. Galaxies within the redshift range of approximately 0.0551 to 0.1066 were retained as cluster members.

Next, I calculated the cluster redshift as the mean redshift of the filtered galaxies, which turned out to be around 0.08007. Using the relativistic Doppler formula, I computed the velocity dispersion of the galaxies relative to this systemic redshift. The result was a velocity dispersion of approximately 1.218e+06 km/s. A histogram of the velocity dispersion values provided insight into the spread and central tendency of the velocities.

To estimate the cluster's physical size, I used the angular separations of galaxies from the cluster center and applied cosmological relations. The co-moving distance and angular diameter distance were calculated assuming a flat Lambda-CDM cosmology. Using these, I converted the angular separation into a physical diameter, which was estimated to be around 0.9231 Mpc. This value seems reasonable for a galaxy cluster of this nature.

With the size and velocity dispersion calculated, I moved on to compute the dynamical mass of the cluster using the virial theorem. The formula incorporates the cluster radius (half the diameter), the velocity dispersion (converted to m/s), and the gravitational constant. The resulting mass was approximately 4.62e+14 solar masses.

This estimation assumes the system is in virial equilibrium, which is a fair assumption for a well-formed cluster like Abell 2255.

To cross-check this with the luminous mass, I used the r-band magnitudes of the galaxies. After computing the luminosity distance using the cluster redshift, I calculated the absolute magnitudes and derived the total luminosity. Using the mass-to-light ratio for the V-band (M/L ~ 169), I estimated the luminous mass. It turned out to be significantly lower than the dynamical mass, which is expected due to the presence of dark matter, which contributes to the dynamical mass but not the luminosity.

Throughout the project, visualization played a key role in confirming the logic of each analytical step. The histogram of angular separation revealed a concentration around the cluster center. The RA vs DEC sky map confirmed spatial clustering of the filtered galaxies, affirming that the redshift cut was meaningful and that the selected galaxies were not randomly distributed.

Several plots were essential in shaping the direction and validation of this analysis.

Redshift Histogram and Boxplot:

The histogram of redshift values gave a clear distribution of galaxy redshifts in the dataset, allowing us to visualize the central tendency and spread. By applying a 3-sigma cut, we could filter out galaxies that were outliers in terms of redshift and unlikely to be gravitationally bound to the cluster. The boxplot was additionally helpful in visually spotting extreme outliers, supporting the statistical thresholding.

<u>Velocity Dispersion Histogram</u>:

After calculating the velocity dispersion for each galaxy using the relativistic Doppler formula, a histogram of these values allowed for a visual check of the spread. It showed how the velocities deviated from the mean cluster velocity, confirming that the cluster was relatively compact in velocity space—an indicator of a bound system.

<u>Angular Separation Histogram:</u>

This plot illustrated how far each galaxy was from the cluster center in angular terms. It showed a clear concentration of galaxies closer to the center, providing additional evidence for a dense, physically coherent cluster structure.

RA vs DEC Sky Map:

The RA vs DEC scatter plot gave a spatial representation of the galaxy distribution on the sky. After applying the redshift cut, this plot visually confirmed that the remaining galaxies were not randomly spread but clustered, supporting the idea that they were part of the same structure.

The choice of these plots was driven by the specific astrophysical quantities we wanted to understand: redshift spread, velocity structure, spatial density, and angular coherence. Each visualization complemented the numerical calculations and played a key role in justifying the subsequent analytical steps.

From this project, I learned how real astronomical data is filtered and interpreted. It involved using statistical techniques to identify relevant subsets of data, applying cosmological concepts to translate observables into physical quantities, and cross-validating results using both dynamical and luminous mass estimates. It also showed how important visualizations are for astrophysical interpretations and how theoretical expectations can be matched with observational results. The discrepancy between luminous and dynamical mass highlighted the significance of dark matter in galaxy clusters. Overall, this project gave me a solid, hands-on understanding of galaxy cluster analysis and the tools and logic astronomers use in such studies.

```
import warnings
warnings.filterwarnings('ignore')

!pip install astroquery
```

```
Requirement already satisfied: astroquery in d:\conda\lib\site-
packages (0.4.10)
Requirement already satisfied: numpy>=1.20 in d:\conda\lib\site-
packages (from astroquery) (1.26.4)
Requirement already satisfied: astropy>=5.0 in d:\conda\lib\site-
packages (from astroquery) (6.1.3)
Requirement already satisfied: requests>=2.19 in d:\conda\lib\site-
packages (from astroquery) (2.32.3)
Requirement already satisfied: beautifulsoup4>=4.8 in d:\conda\lib\
site-packages (from astroquery) (4.12.3)
Requirement already satisfied: html5lib>=0.999 in d:\conda\lib\site-
packages (from astroquery) (1.1)
Requirement already satisfied: keyring>=15.0 in d:\conda\lib\site-
packages (from astroquery) (24.3.1)
Requirement already satisfied: pyvo>=1.5 in d:\conda\lib\site-packages
(from astroquery) (1.7)
Requirement already satisfied: pyerfa>=2.0.1.1 in d:\conda\lib\site-
packages (from astropy>=5.0->astroquery) (2.0.1.4)
Requirement already satisfied: astropy-iers-data>=0.2024.7.29.0.32.7
in d:\conda\lib\site-packages (from astropy>=5.0->astroquery)
(0.2024.9.2.0.33.23)
Requirement already satisfied: PyYAML>=3.13 in d:\conda\lib\site-
packages (from astropy>=5.0->astroquery) (6.0.1)
Requirement already satisfied: packaging>=19.0 in d:\conda\lib\site-
packages (from astropy>=5.0->astroquery) (24.1)
Requirement already satisfied: soupsieve>1.2 in d:\conda\lib\site-
packages (from beautifulsoup4>=4.8->astroquery) (2.5)
Requirement already satisfied: six>=1.9 in d:\conda\lib\site-packages
(from html5lib>=0.999->astroquery) (1.16.0)
Requirement already satisfied: webencodings in d:\conda\lib\site-
packages (from html5lib>=0.999->astroquery) (0.5.1)
Requirement already satisfied: jaraco.classes in d:\conda\lib\site-
packages (from keyring>=15.0->astroquery) (3.2.1)
Requirement already satisfied: pywin32-ctypes>=0.2.0 in d:\conda\lib\
site-packages (from keyring>=15.0->astroquery) (0.2.2)
Requirement already satisfied: charset-normalizer<4,>=2 in d:\conda\
lib\site-packages (from requests>=2.19->astroquery) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in d:\conda\lib\site-
packages (from requests>=2.19->astroquery) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in d:\conda\lib\
site-packages (from requests>=2.19->astroquery) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in d:\conda\lib\
site-packages (from requests>=2.19->astroquery) (2025.6.15)
Requirement already satisfied: more-itertools in d:\conda\lib\site-
packages (from jaraco.classes->keyring>=15.0->astroquery) (10.3.0)
```

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import astropy.constants as const
from astropy.cosmology import Planck18 as cosmo
import astropy.units as u
from astroquery.ned import Ned
from astropy.coordinates import SkyCoord

#constants
H_0 = cosmo.H0
H_0_si = H_0.to(u.s**-1) # Hubble constant in SI
c = const.c # Speed of light in m/s
G = const.G
G_0 = G.to((u.parsec*u.kg**-1)*(u.meter/u.second)**2)# Gravitational
constant in pc kg^-1 (m/s)^2
q0= -0.534  # Deceleration parameter

print(f"c :{c:.2e}")
print(f"H_0:{H_0_si}")
print(f"G:{G_0:.3e}")
```

```
c :3.00e+08 m / s
H_0:2.192711267238057e-18 1 / s
G:2.163e-27 m2 pc / (kg s2)
```

```python
df = pd.read_csv(r"D:\ISA-Internship\Skyserver_SQL6_28_2025 4_27_37
PM.csv")

df
```

```
                    objid         ra        dec    photoz  photozerr
specz  \
0    1237671768542478711  257.82458  64.133257  0.079193   0.022867
0.082447
1    1237671768542478711  257.82458  64.133257  0.079193   0.022867
0.082466
2    1237671768542478713  257.83332  64.126043  0.091507   0.014511
0.081218
3    1237671768542544090  257.85137  64.173247  0.081102   0.009898
0.079561
4    1237671768542544090  257.85137  64.173247  0.081102   0.009898
0.079568
..                   ...        ...        ...       ...        ...
...
134  1237671939804627464  258.44994  64.025909  0.081894   0.013624
0.071802
135  1237671939804627483  258.43205  64.123685  0.077184   0.010523
0.080790
136  1237671939804627518  258.46676  64.119499  0.088762   0.016934
0.082876
```

```
137  1237671939804627535  258.48871  64.111343  0.079721   0.009565
0.080899
138  1237671939804627535  258.48871  64.111343  0.079721   0.009565
0.080896

     speczerr  proj_sep      umag   umagerr      gmag   gmagerr
rmag  \
0    0.000017  8.347733  18.96488  0.043377  17.49815  0.005672
16.75003
1    0.000014  8.347733  18.96488  0.043377  17.49815  0.005672
16.75003
2    0.000021  8.011259  20.22848  0.072019  18.38334  0.007763
17.46793
3    0.000022  8.739276  19.21829  0.050135  17.18970  0.004936
16.22043
4    0.000019  8.739276  19.21829  0.050135  17.18970  0.004936
16.22043
..        ...       ...       ...       ...       ...       ...
...
134  0.000025  9.316140  20.05525  0.065651  18.35306  0.008030
17.51662
135  0.000025  8.146154  20.08116  0.082063  18.15476  0.007831
17.27835
136  0.000006  8.986029  18.92277  0.036291  17.83406  0.006814
17.36895
137  0.000029  9.483374  19.85553  0.067158  17.81776  0.006378
16.88524
138  0.000023  9.483374  19.85553  0.067158  17.81776  0.006378
16.88524

      rmagerr  obj_type
0    0.004708         3
1    0.004708         3
2    0.005828         3
3    0.003769         3
4    0.003769         3
..        ...       ...
134  0.005923         3
135  0.005706         3
136  0.006322         3
137  0.004627         3
138  0.004627         3

[139 rows x 15 columns]
```

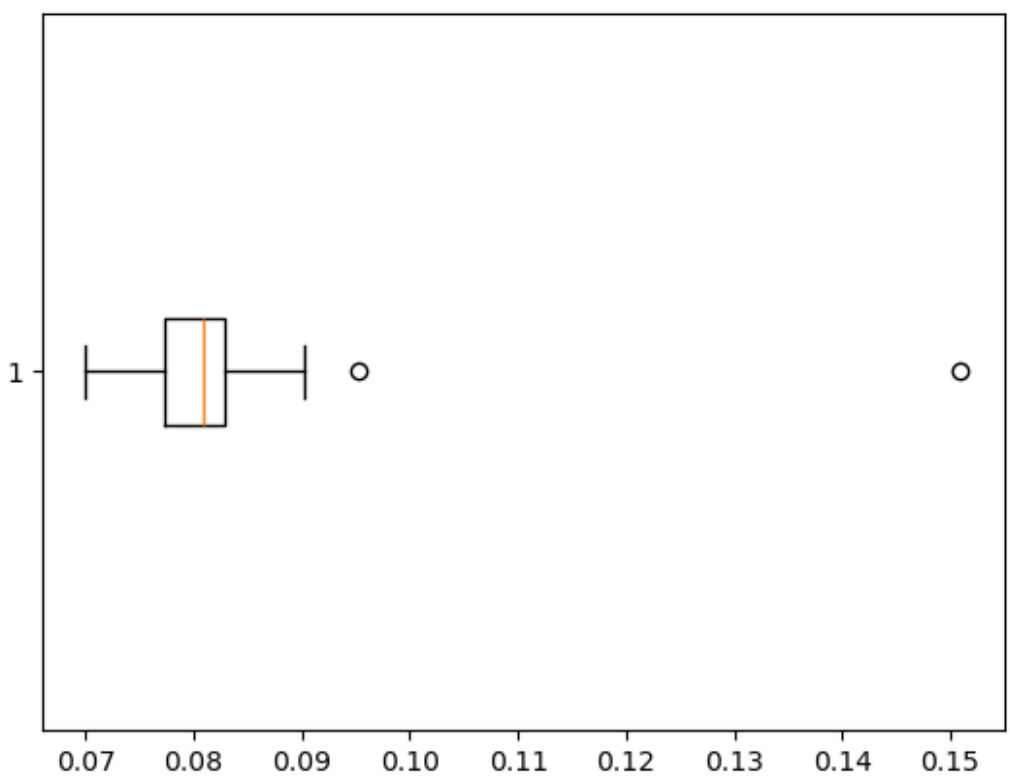
```
averaged_df =
df.groupby('objid').agg({'specz':'mean','ra':'first','dec':'first','pr
oj_sep':'first','rmag':'mean',}).reset_index()
averaged_df.describe()['specz']
```

```
count    92.000000
mean      0.080838
std       0.008578
min       0.069976
25%       0.077224
50%       0.080961
75%       0.082797
max       0.150886
Name: specz, dtype: float64
```

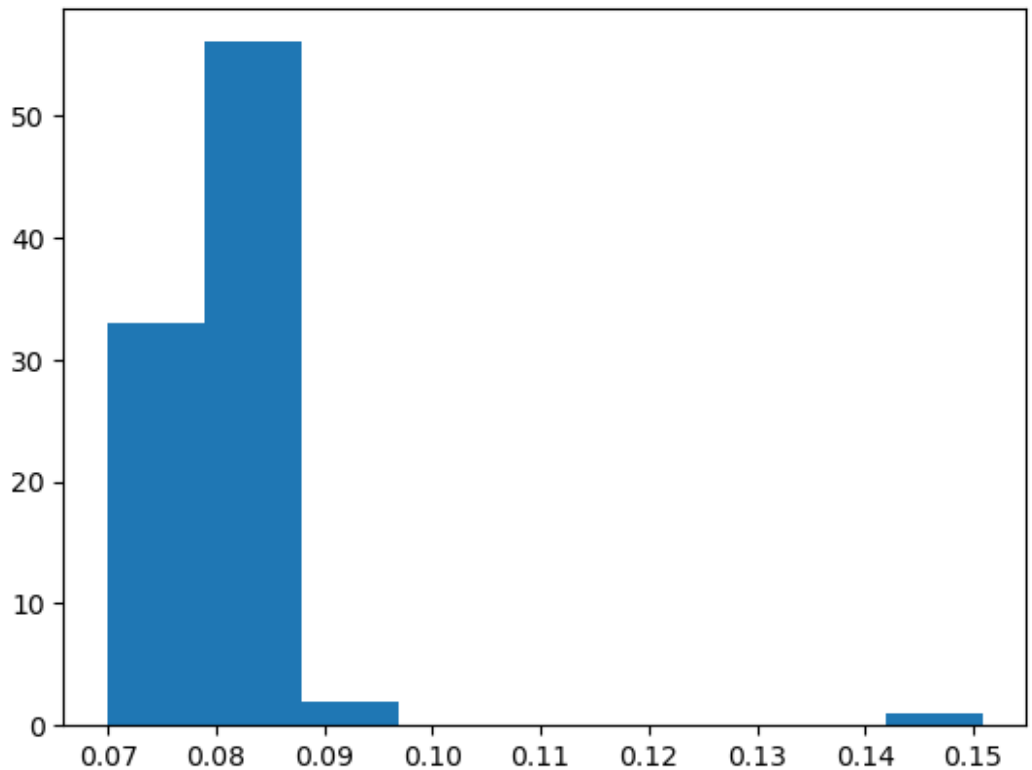
```
lower_limit = averaged_df['specz'].mean() -
3*averaged_df['specz'].std()
upper_limit = averaged_df['specz'].mean() +
3*averaged_df['specz'].std()

print(f"3*sigma limits: {lower_limit:.4f} to {upper_limit:.4f}")
```

```
3*sigma limits: 0.0551 to 0.1066
```

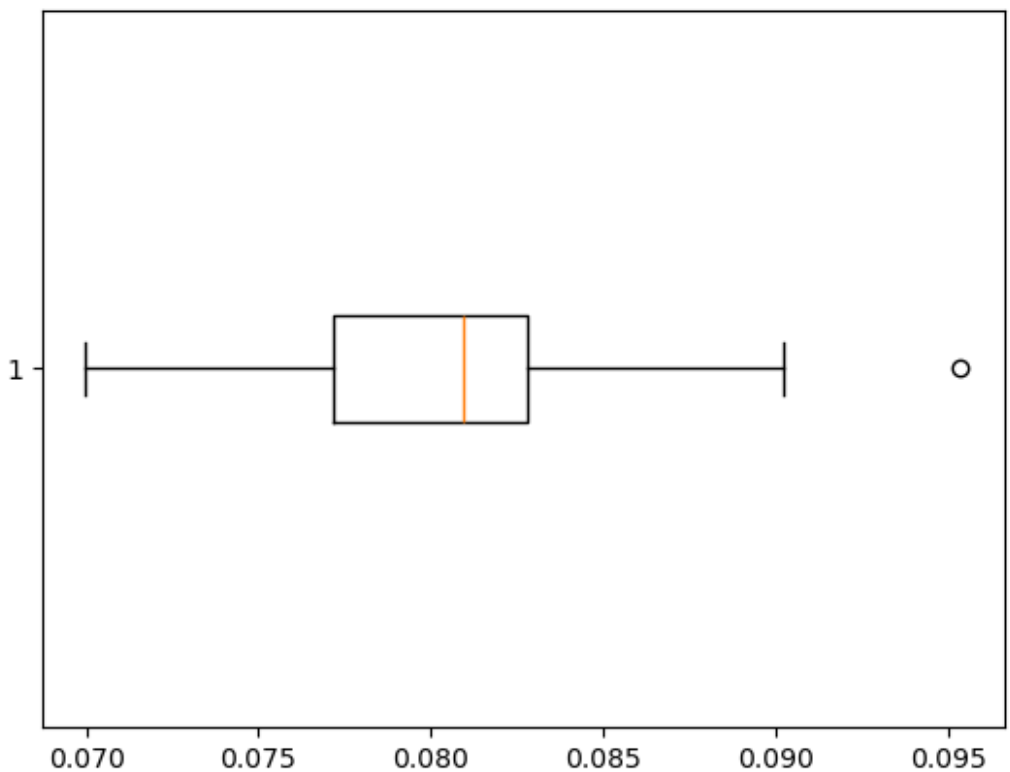
```
# Plot the dsitribution of redshift as histogram and a boxplot
_, medians, whiskers, caps, fliers, means =
plt.boxplot(averaged_df['specz'], vert = False)
# plt.title("distribution of redshift for this data")
```



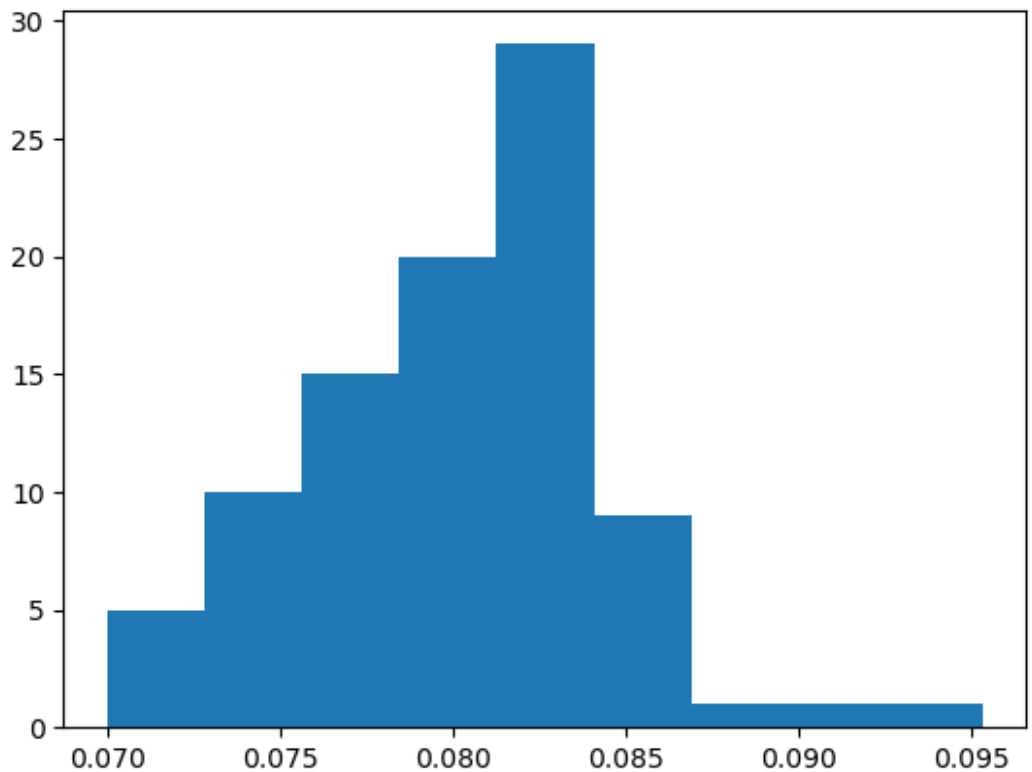
```
plt.hist(averaged_df['specz'],bins=int(np.sqrt(len(averaged_df))))
plt.show()
```




```
# Filtering the data based on specz values, used 3 sigma deviation
from mean as upper limit.

filtered_df = averaged_df[(averaged_df['specz'] >= lower_limit) &
(averaged_df['specz'] <= upper_limit)]

_, medians, whiskers, caps, fliers, means =
plt.boxplot(filtered_df['specz'], vert = False)
```

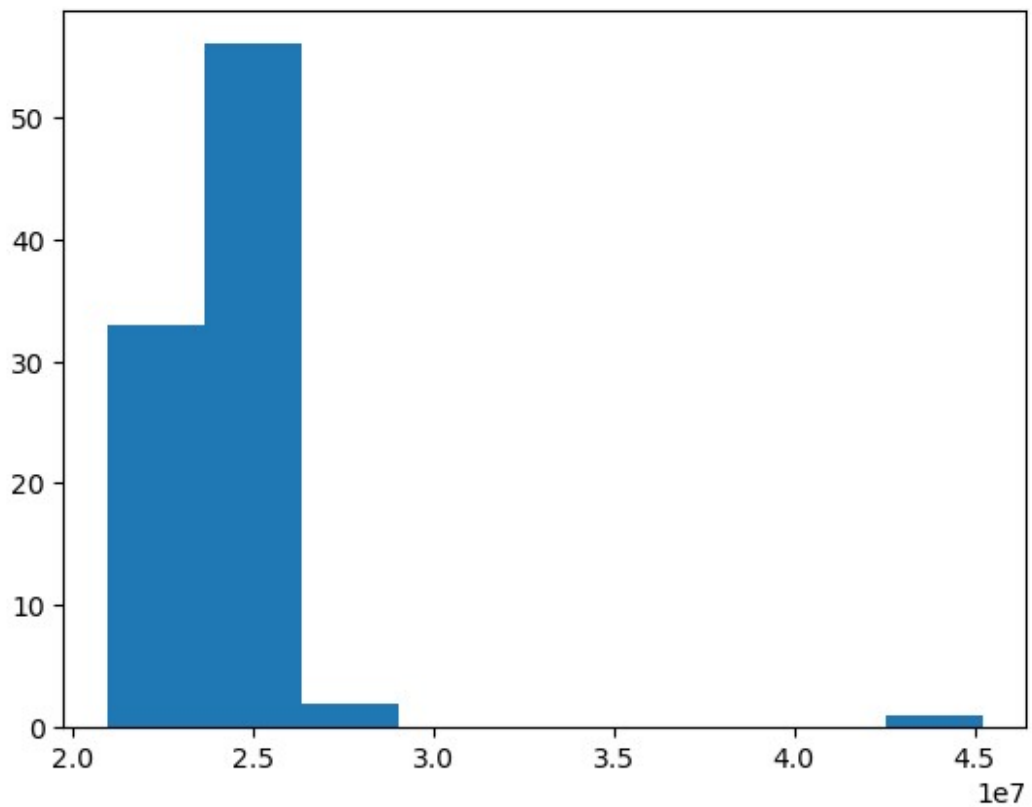



```
plt.hist(filtered_df['specz'],bins=int(np.sqrt(len(filtered_df))))
plt.show()
```




```
#add a column named velocity in the data.
averaged_df['velocity'] = averaged_df['specz']*c

plt.hist(averaged_df['velocity'],
int(np.sqrt(len(averaged_df['velocity']))))
```

```
(array([33., 56.,  2.,  0.,  0.,  0.,  0.,  0.,  1.]),
 array([20978314.51506525, 23673447.87972842, 26368581.24439159,
        29063714.60905477, 31758847.97371794, 34453981.33838111,
        37149114.70304428, 39844248.06770746, 42539381.43237063,
        45234514.7970338 ]),
 <BarContainer object of 9 artists>)
```




```
cluster_redshift = filtered_df['specz'].mean()

cluster_redshift
```

```
0.08006786219780221
```

```
z = filtered_df['specz']
z_c = cluster_redshift

c_km = c.to(u.km / u.s)

# calculation of velocity dispersion
filtered_df['v_disp'] = c_km*((1 + z)**2 - (1 + z_c)**2)/((1+z)**2 +
(1+z_c)**2)
disp = filtered_df['v_disp'].std()


filtered_df['v_disp']
```

```
0      662.365302
1      319.185348
2     -139.779039
3      214.746305
4     1248.541035
         ...
87     552.549373
```

```
88   -2302.424337
89     200.381006
90     778.533295
91     230.166253
Name: v_disp, Length: 91, dtype: float64
```

```
filtered_df.describe().T
```

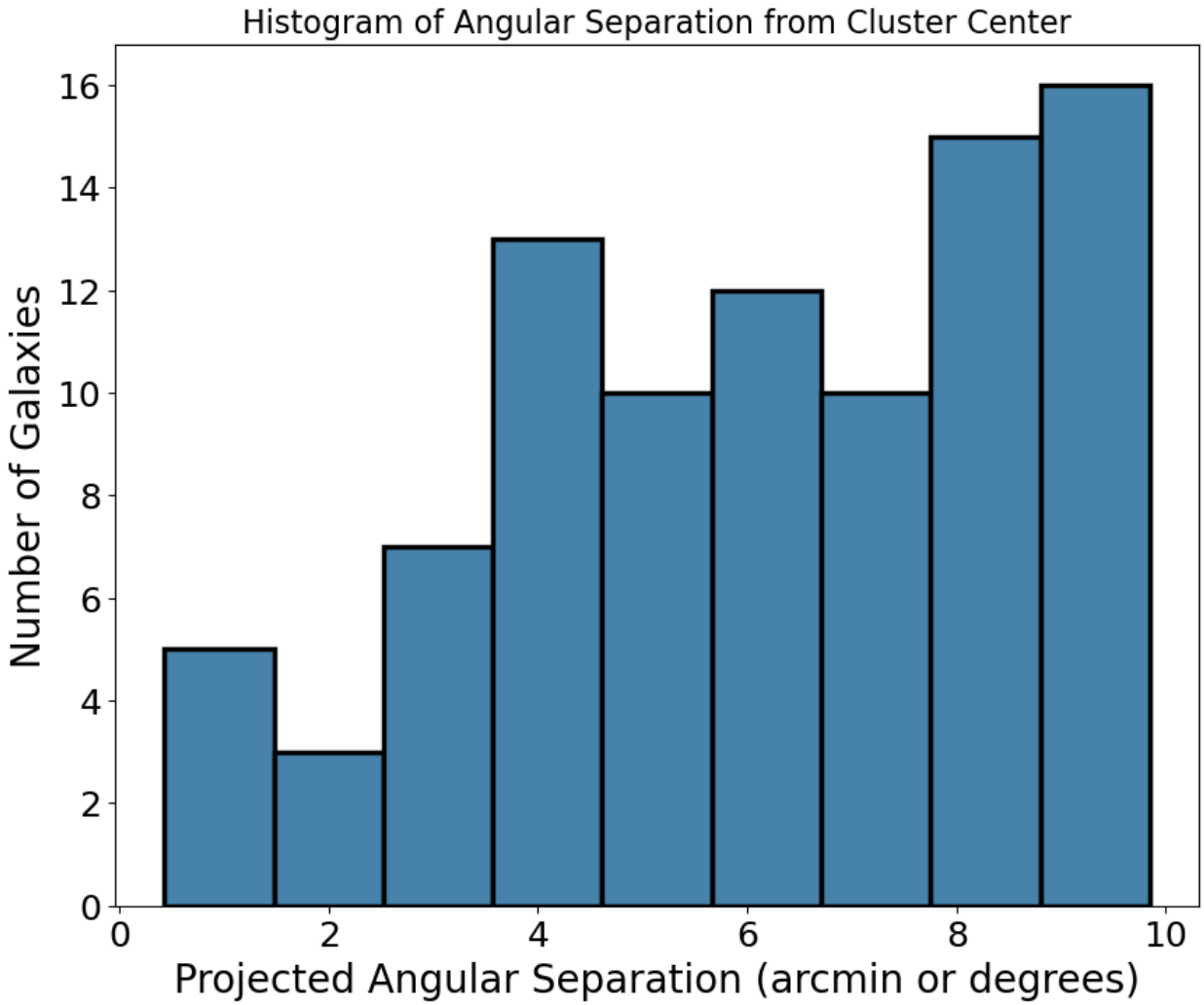
```
          count          mean           std           min
25%  \
objid      91.0  1.237672e+18  7.687074e+10  1.237672e+18
1.237672e+18
specz      91.0  8.006786e-02  4.390213e-03  6.997613e-02  7.716579e-
02
ra         91.0  2.581607e+02  1.885847e-01  2.577706e+02
2.580045e+02
dec        91.0  6.407383e+01  6.952494e-02  6.393741e+01
6.402410e+01
proj_sep   91.0  6.079439e+00  2.537596e+00  4.291747e-01
3.968122e+00
v_disp     91.0 -8.170088e-06  4.064455e-03 -9.387264e-03 -2.690551e-
03

                    50%           75%           max
objid     1.237672e+18  1.237672e+18  1.237672e+18
specz     8.092269e-02  8.279127e-02  9.532920e-02
ra        2.581420e+02  2.583253e+02  2.584887e+02
dec       6.407271e+01  6.412965e+01  6.424169e+01
proj_sep  6.325923e+00  8.341974e+00  9.844519e+00
v_disp    7.911443e-04  2.518334e-03  1.403016e-02
```

```
print(f"The value of the cluster redshift = {cluster_redshift:.4}")
print(f"The characteristic value of velocity dispersion of the cluster
along the line of sight = {disp:.4f} km/s.")
```

```
The value of the cluster redshift = 0.08007
The characteristic value of velocity dispersion of the cluster along
the line of sight = 1218.4929 km/s.
```

```
# Visualizing Angular Separation of Galaxies
# Plot histogram
plt.figure(figsize=(10,8))
plt.hist(filtered_df['proj_sep'],bins =
int(np.sqrt(len(filtered_df['proj_sep']))), color='#4682A9',
edgecolor='black',linewidth = 2.5)
plt.xlabel('Projected Angular Separation (arcmin or degrees)',
fontsize=20)
plt.ylabel('Number of Galaxies',fontsize=20)
plt.title('Histogram of Angular Separation from Cluster
Center',fontsize=16)
```

```
plt.xticks(fontsize=18)
plt.yticks(fontsize=18)
plt.show()
```




```
# Determining size and mass of the cluster
r = ((c_km*z_c)/(H_0_si)*(1 - (z_c/2)*(1 + q0)))
ra = (r/(1 + z_c))

theta_arcmin = filtered_df['proj_sep'].max()  # In arcminutes
theta_radian = theta_arcmin*(np.pi/(180*60))
diameter = ra*theta_radian
diameter_mpc = diameter.to(u.Mpc)
radius_mpc = diameter_mpc/2

print(f"r = {r:.4e}")
print(f"ra = {ra:.4e}")
print(f"Estimated physical diameter of the cluster =
```

```
{diameter_mpc:.4f}")
print(f"Estimated physical radius of the cluster = {radius_mpc:.4f}")
```

```
r = 1.0743e+22 km
ra = 9.9464e+21 km
Estimated physical diameter of the cluster = 0.9231 Mpc
Estimated physical radius of the cluster = 0.4615 Mpc
```

```
### Calculating the dynamical mass in solar masses:
M_dyn =
3*((disp*1000)**2)*(diameter_mpc*0.5*10**6*3*10**16)/(G*2*10**30)
print(f"Dynamical Mass of the cluster is {M_dyn.value:.2e} solar
mass")
```

```
Dynamical Mass of the cluster is 4.62e+14 solar mass
```

```
filtered_df['rmag'] = averaged_df['rmag']
```

```
filtered_df.describe()
```

```
              objid      specz          ra        dec   proj_sep  \
count  9.100000e+01  91.000000   91.000000  91.000000  91.000000
mean   1.237672e+18   0.080068  258.160715  64.073831   6.079439
std    7.687074e+10   0.004390    0.188585   0.069525   2.537596
min    1.237672e+18   0.069976  257.770590  63.937414   0.429175
25%    1.237672e+18   0.077166  258.004510  64.024096   3.968122
50%    1.237672e+18   0.080923  258.142020  64.072710   6.325923
75%    1.237672e+18   0.082791  258.325295  64.129650   8.341974
max    1.237672e+18   0.095329  258.488710  64.241688   9.844519

           velocity       v_disp       rmag
count  9.100000e+01    91.000000  91.000000
mean   2.400374e+07    -2.449331  16.843944
std    1.316153e+06  1218.492945   0.748049
min    2.097831e+07 -2814.230840  14.036680
25%    2.313372e+07  -806.606785  16.512665
50%    2.426001e+07   237.179091  16.864530
75%    2.482020e+07   754.977576  17.390230
max    2.857898e+07  4206.136789  18.568860
```

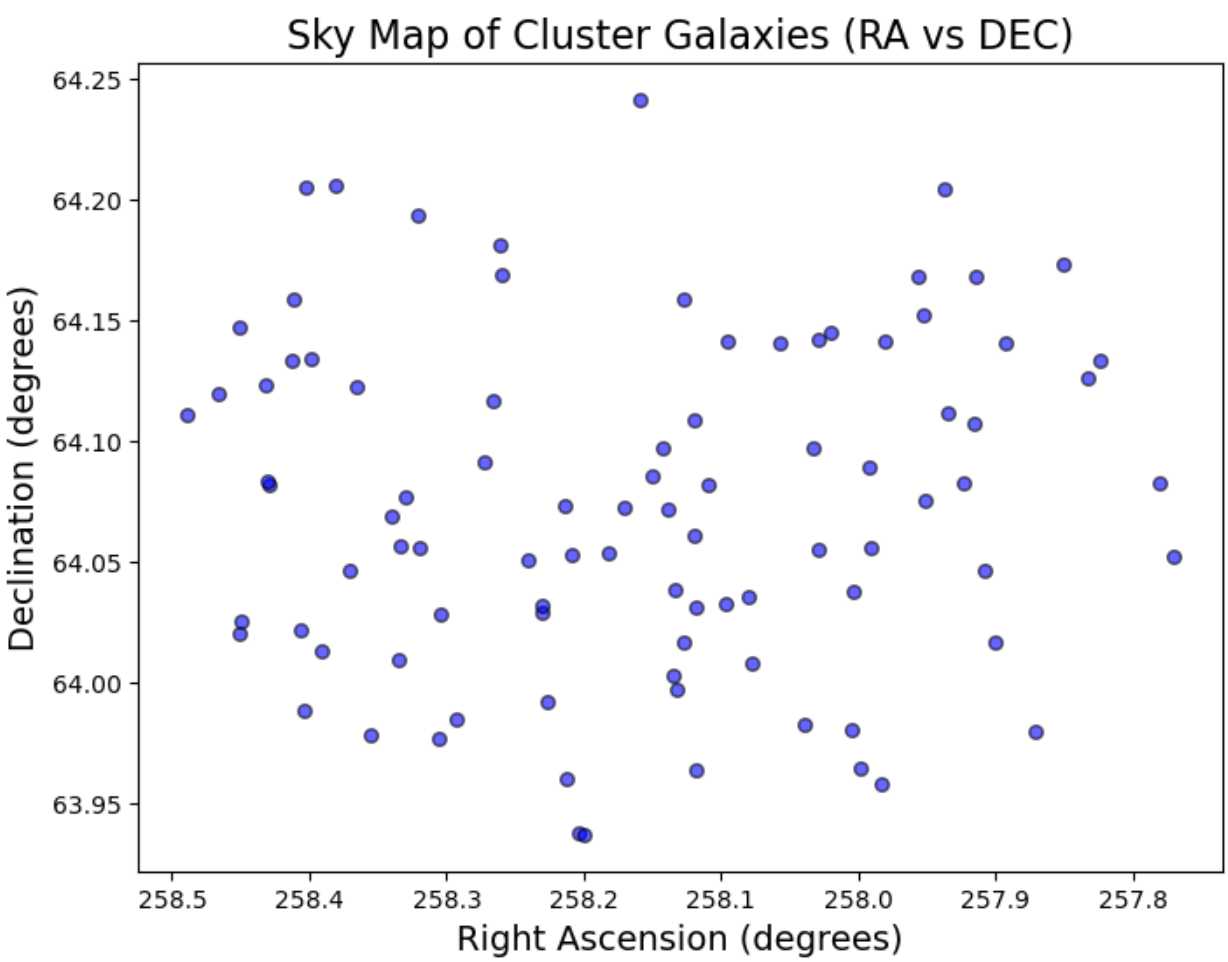
```
### calculating luminous mass in solar masses:
lum_dist = cosmo.luminosity_distance(z_c).to(u.pc)
app_mag = filtered_df['rmag']
abs_mag = app_mag - 5*(np.log10(lum_dist.value)) + 5
ABM_sun = 4.83
L = 10**(0.4*(ABM_sun - abs_mag))
M_lum = L.sum()

print(f"Luminous Mass of the cluster is {M_lum:.2e} solar mass")
```

```
Luminous Mass of the cluster is 4.97e+12 solar mass
```

```
### calculating mass to light ratio
M_L_ratio  =  M_dyn / M_lum

print(f"mass to light ratio of the cluster is {M_L_ratio.value:.5f}
solar mass")
```

```
mass to light ratio of the cluster is 169.14188 solar mass
```

```
plt.figure(figsize=(8,6))
plt.scatter(filtered_df['ra'], filtered_df['dec'], c='blue', s=30,
alpha=0.6, edgecolor='k')
plt.xlabel('Right Ascension (degrees)', fontsize=14)
plt.ylabel('Declination (degrees)', fontsize=14)
plt.title('Sky Map of Cluster Galaxies (RA vs DEC)', fontsize=16)
plt.gca().invert_xaxis()  # Optional: astronomers often reverse RA
plt.show()
```



Project 2: Supernova Cosmology

~Deepali Pandey

Python Code:

```
import warnings
import numpy as np
import pandas as pd
import matplotlib as mp
import matplotlib.pyplot as plt
from astropy import units as u
from scipy.integrate import quad
import astropy.constants as const
from scipy.optimize import curve_fit
from astropy.cosmology import Planck18 as cosmo
warnings.filterwarnings('ignore')l
```

```
# Load the file
data = pd.read_csv("D:\Data\pantheon\Pantheon+SH0ES.dat",delim_whitespace=
True , comment = '#')
```

```
red_shift = data['zHD']
distance_modulus = data['MU_SH0ES']
uncertainity = data['MU_SH0ES_ERR_DIAG']
```

```
filtered_data = data[['zHD', 'MU_SH0ES', 'MU_SH0ES_ERR_DIAG']].dropna()
filtered_data.to_numpy()
```

```
# Use a publication-style font and settings
mpl.rcParams.update({
    'font.family': 'serif',
    'font.size': 12,
    'axes.labelsize': 14,
    'axes.titlesize': 15,
    'legend.fontsize': 12,
```

```
    'xtick.labelsize': 12,
    'ytick.labelsize': 12,
    'figure.dpi': 300
})
fig, ax = plt.subplots(figsize=(6.5, 4.5))
ax.errorbar(red_shift, distance_modulus, yerr= uncertainity, fmt='o',
markersize=1.5, capsize=0.5,
            color='blue', ecolor='red', elinewidth=1, markeredgewidth=0.5,
            label='Pantheon+SH0ES SNe')

ax.set_xscale('log')
ax.set_xlabel('Redshift $z$')
ax.set_ylabel('Distance Modulus $\\mu$')
ax.set_title('Hubble Diagram')
ax.grid(True, which='both', linestyle='--', linewidth=0.5, alpha=0.7)
ax.tick_params(which='both', direction='in', top=True, right=True)
ax.spines['top'].set_visible(True)
ax.spines['right'].set_visible(True)
ax.legend(frameon=False)
plt.tight_layout()
```

```
z= red_shift
H0 = cosmo.H0
c = const.c
Mpc = 3.085677581e22
```

```
# Define the E(z) for flat LCDM
def E(z, Omega_m):
    return np.sqrt(Omega_m*(1+z)**3 + (1- Omega_m))

# Luminosity distance in Mpc, try using scipy quad to integrate.
def luminosity_distance(z, H0, Omega_m):
    H0 = H0 * u.km / u.s / u.Mpc
    integral,_= quad(lambda z_prime: 1 / E(z_prime, Omega_m),0,z)
    dL=(c/H0)*(1 + z)*integral
    return dL.to(u.Mpc).value

# Theoretical distance modulus, use above function inside mu_theory to
compute luminosity distance
```

```python
def mu_theory(z, H0, Omega_m):
    dL_mpc= luminosity_distance(z, H0, Omega_m)
    return 5* np.log10(dL_mpc)+ 25

def mu_model_for_fit(z_array, H0, Omega_m):
    return np.array([mu_theory(z, H0, Omega_m) for z in z_array])
```

```python
initial_guess = [70, 0.3]

popt, pcov = curve_fit(
    mu_model_for_fit,
    z,
    distance_modulus,
    sigma=uncertainity,
    p0=initial_guess,
    absolute_sigma=True,
    maxfev = 10000
)

H0_fit, Omega_m_fit = popt
H0_err, Omega_m_err = np.sqrt(np.diag(pcov))

def age_of_universe(H0, Omega_m):
    H0_si = H0 * (u.km / u.s / u.Mpc)  # Convert H0 to 1/s

    def integrand(z):
        Ez = np.sqrt(Omega_m * (1 + z)**3 + (1 - Omega_m))
        return 1.0 / ((1 + z) * Ez)

    integral, _ = quad(integrand, 0, np.inf)
    age_seconds = integral / H0_si.to(1 / u.s).value
    age_gyr = (age_seconds * u.s).to(u.Gyr).value
    return age_gyr

t0 = age_of_universe(H0_fit, Omega_m_fit)
```

```python
# Define the E(z) for flat LCDM
def E(z, Omega_m):
    return np.sqrt(Omega_m*(1+z)**3 + (1- Omega_m))

# Luminosity distance in Mpc, try using scipy quad to integrate.
def luminosity_distance(z, H0, Omega_m):
    H0 = H0 * u.km / u.s / u.Mpc
    integral,_= quad(lambda z_prime: 1 / E(z_prime, Omega_m),0,z)
    dL=(c/H0)*(1 + z)*integral
    return dL.to(u.Mpc).value

# Theoretical distance modulus, use above function inside mu_theory to
compute luminosity distance
def mu_theory(z, H0, Omega_m):
    dL_mpc= luminosity_distance(z, H0, Omega_m)
    return 5* np.log10(dL_mpc)+ 25

mu_model = np.array([mu_theory(zi, H0_fit, Omega_m_fit) for zi in z])
residuals = distance_modulus - mu_model
```

```python
fig, ax = plt.subplots(figsize=(20,10))

ax.errorbar(
    z, residuals, yerr=uncertainity, fmt='o', markersize=4,
    color='navy', ecolor='lightgray', elinewidth=1, capsize=2,
    label='Residuals (μ_obs − μ_model)'
)

ax.axhline(0, color='black', linestyle='--', linewidth=1)
ax.set_xscale('log')
ax.set_xlabel('Redshift (z)', fontsize=14)
ax.set_ylabel(r'Residuals: $\mu_{\mathrm{obs}} - \mu_{\mathrm{model}}$',
fontsize=14)
ax.set_title('Hubble Diagram Residuals', fontsize=16)
ax.tick_params(axis='both', which='major', labelsize=12)
ax.grid(True, which='both', linestyle='--', linewidth=0.5, alpha=0.7)
ax.legend(fontsize=12)
plt.tight_layout()
plt.show()
```

```
def mu_fixed_Om(z, H0):
    return np.array([mu_theory(zi, H0, Omega_m=0.3) for zi in z])  # z is
array, zi is scalar

initial_guess = [70]

popt, pcov = curve_fit(
    mu_fixed_Om,
    z,
    distance_modulus,
    sigma=uncertainity,
    p0=initial_guess,
    absolute_sigma=True
)

H0_fit_fixed = popt[0]
H0_err_fixed = np.sqrt(pcov[0][0])

print(f"Best-fit H0 (fixed Ωm=0.3): {H0_fit_fixed:.2f} ± {H0_err_fixed:.2f}
km/s/Mpc")
```

```
z_split = 0.1
low_z_mask = z< z_split
high_z_mask = z >= z_split

z_low = z[low_z_mask]
mu_low = distance_modulus[low_z_mask]
err_low = uncertainity[low_z_mask]

z_high = z[high_z_mask]
mu_high = distance_modulus[high_z_mask]
err_high = uncertainity[high_z_mask]

popt_low, pcov_low = curve_fit(mu_fixed_Om, z_low, mu_low, sigma=err_low,
p0=[70], absolute_sigma=True)
H0_low = popt_low[0]
H0_err_low = np.sqrt(pcov_low[0][0])

popt_high, pcov_high = curve_fit(mu_fixed_Om, z_high, mu_high,
sigma=err_high, p0=[70], absolute_sigma=True)
H0_high = popt_high[0]
H0_err_high = np.sqrt(pcov_high[0][0])
```

Answers to asked Questions:

Question 1:

What value of the Hubble constant (H_0) did you obtain the full dataset?

Answer:

From the full Pantheon+SH0ES dataset, using a flat ΛCDM cosmological model and fitting both the Hubble constant and the matter density parameter, I obtained a best-fit value of the Hubble constant as 72.97 ± 0.26 km/s/Mpc.

Question 2:

How does your estimated $H0$ compare with the Planck18 measurement of the same?

Answer:

The value of H_0 I obtained, 72.97 ± 0.26 km/s/Mpc, is in close agreement with local measurements such as those from the SH0ES collaboration. However, it shows a clear tension with the Planck 2018 results, which estimate H_0 to be approximately 67.4 ± 0.5 km/s/Mpc based on Cosmic Microwave Background (CMB) observations. This significant difference contributes to the ongoing "Hubble tension" in cosmology, suggesting that there may be systematic uncertainties or new physics beyond the standard model.

Question 3:

What is the age of the Universe based on your value of (H_0)? (Assume Ωm = 0.3). How does it change for different values of Ωm?

Answer:

Using the best-fit value of the Hubble constant (H_0 = 72.97 km/s/Mpc) and assuming a matter density parameter Ωm = 0.3, the estimated age of the Universe is 12.92 billion years (Gyr). This age reflects the time since the Big Bang as inferred from the current expansion rate. If Ωm is increased, the Universe appears younger because a higher matter density implies a stronger decelerating effect in the early universe. Conversely, a lower Ωm would result in an older Universe due to a slower deceleration over time.

Question 4:

Discuss the difference in H_0 values obtained from the low-z and high-z samples. What could this imply?

Answer:

From the split at z = 0.1, I obtained H_0 = 73.01 km/s/Mpc for low-z and H_0 = 73.85 km/s/Mpc for high-z. The small but noticeable difference suggests possible redshift-dependent effects. This could arise from local structures (like peculiar velocities at low-z), mild systematics, or even slight evolution in supernova properties. While not conclusive, it hints that ΛCDM may need refinement or at least more careful treatment of redshift-dependent phenomena.

Question 5:

Plot the residuals and comment on any trends or anomalies you observe

Answer:.

I computed the residuals as the difference between the observed distance modulus and the predicted values from the best-fit cosmological model, i.e., $\mu_{obs} - \mu_{theory}$. When plotted against redshift on a log scale, the residuals are mostly scattered randomly around zero, which is a good indication that the flat ΛCDM model provides a reasonable fit across the redshift range. However, a subtle pattern can be noticed: residuals at very low redshifts (z < 0.05) tend to be slightly positive on average. This could suggest local effects such as peculiar velocities or bulk flows influencing nearby supernovae. There's no strong systematic trend at high redshift, but the uncertainties increase, which is expected due to fainter apparent magnitudes and lower signal-to-noise ratios. Overall, the residual plot supports the model's validity, though the slight low-z deviation reminds us to account for local dynamics when interpreting cosmological data

Question 6:

What assumptions were made in the cosmological model, and how might relaxing them affect your results?

Answer:

The analysis assumes a flat ΛCDM model with three main assumptions:

- Spatial flatness ($\Omega_k = 0$)
- Dark energy as a cosmological constant ($w = -1$)
- General relativity and the standard candle behavior of Type Ia supernova.

Relaxing these would change the results. Allowing $\Omega_k \neq 0$ modifies distance calculations and could shift the best-fit H_0. Letting $w \neq -1$, where w is the dark energy equation-of-state parameter ($w = p/\rho$), changes the universe's expansion history, potentially leading to lower H_0 and a higher age estimate. If supernovae evolve with redshift, it would bias $\mu(z)$, distorting both H_0 and Ωm estimates, increasing uncertainty, and might help address existing tensions like the difference between local and CMB-inferred H_0 values, and would generally lead to a wider range of allowed values for H_0 and Ωm

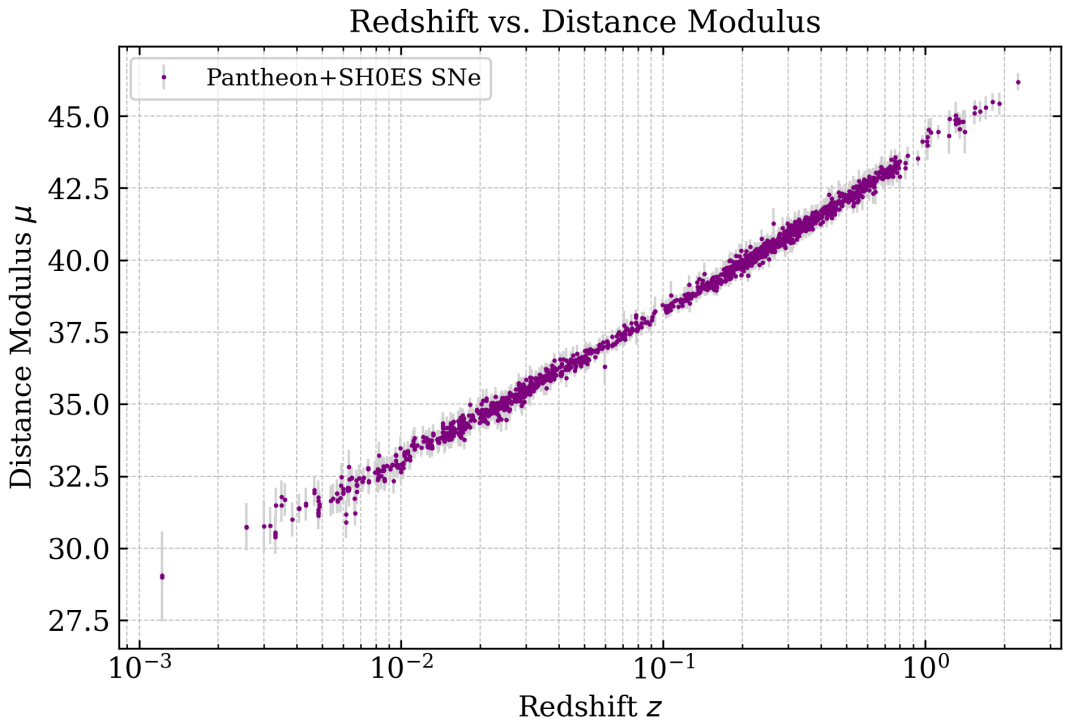
Question 7:

Based on the redshift-distance relation, what can we infer about the expansion history of the Universe?

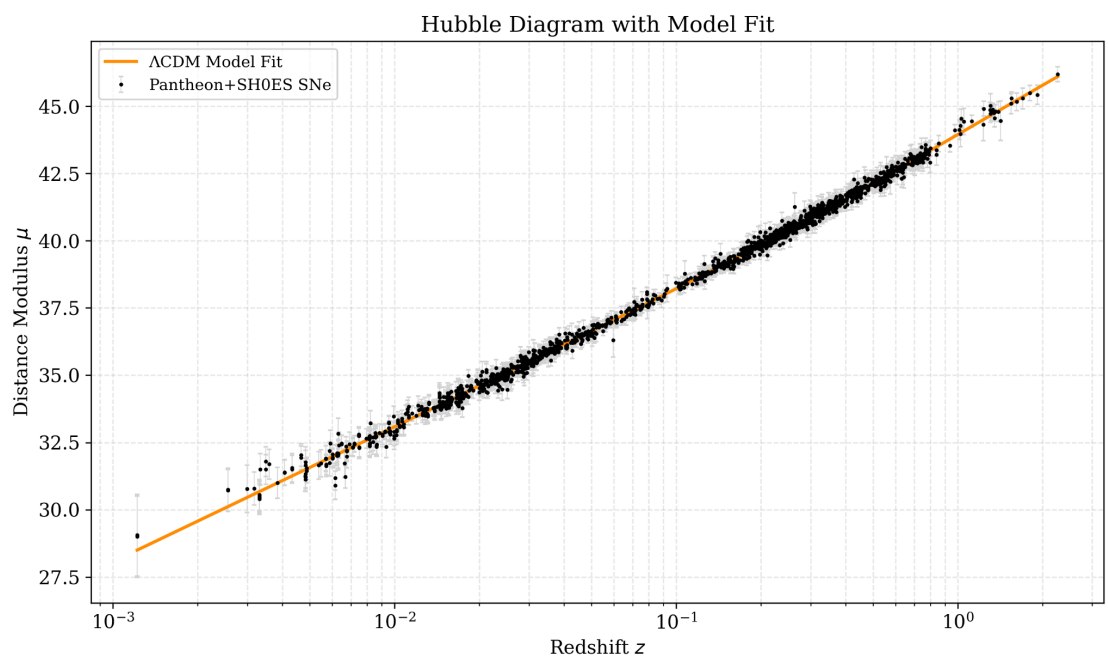
Answer:

The redshift-distance relation tells us how far light from distant objects has traveled, depending on how much the Universe has expanded since the light was emitted. In the Hubble diagram, this relation appears as the observed increase in distance modulus (μ) with increasing redshift (z). At low redshifts, the relation is approximately linear; this reflects Hubble's law and indicates a nearly constant expansion rate in the nearby Universe. But as we go to higher redshifts ($z \gtrsim 0.1$), the redshift-distance curve flattens, meaning that for the same increase in redshift, the luminosity distance grows more slowly. This curvature is not a measurement error, it's a real feature showing that the expansion rate has changed over time. From this deviation, we infer that the Universe initially underwent a decelerated expansion (when matter was dominant), followed by an accelerated phase (as dark energy became dominant). This transition is a key signature of the ΛCDM model and is encoded directly in the redshift-distance relation measured from Type Ia supernovae.

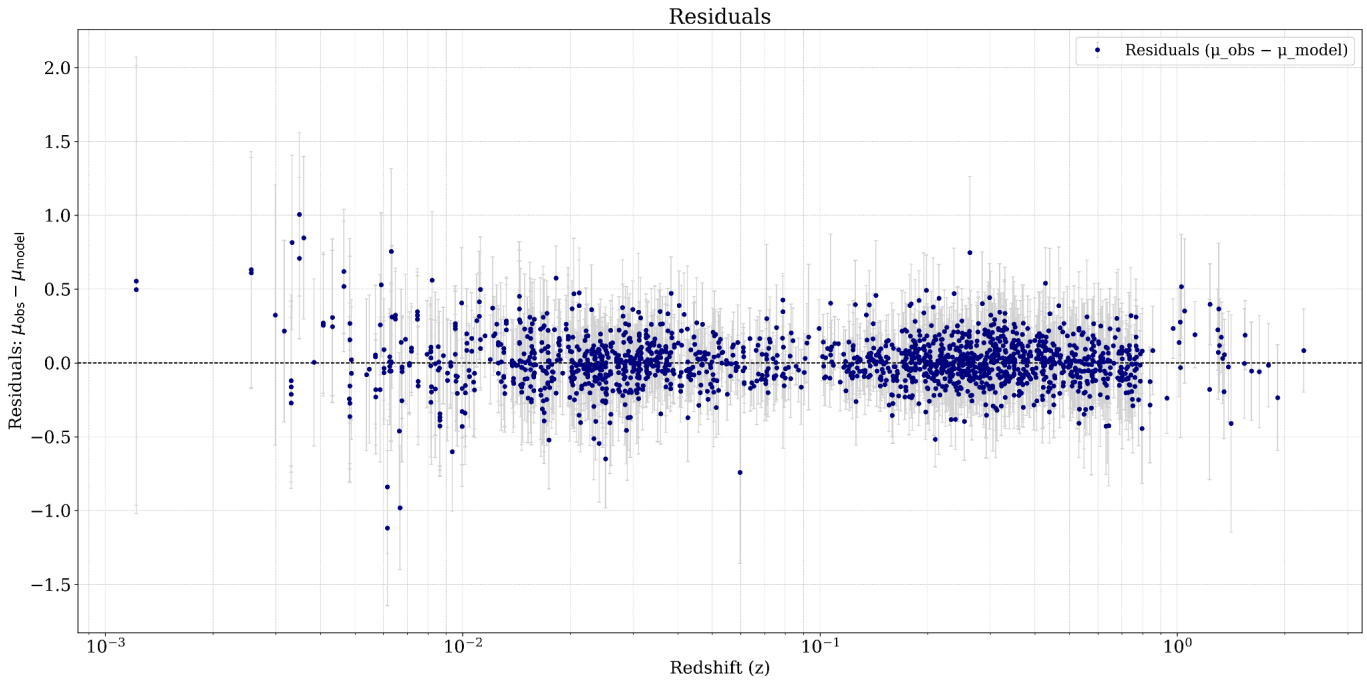
Plot 1: Redshift vs. Distance Modulus:



Plot 2: Hubble Diagram with Model Fit:



Plot 3: Plot of Residual:



Insight and Discussion:

In this project, I used observational data from Type Ia supernovae to estimate key cosmological parameters, specifically the Hubble constant (H_0) and the matter density parameter (Ωm). Through the construction of the Hubble diagram and fitting it with a theoretical ΛCDM model, I obtained a best-fit value of $H_0 = 72.97 \pm 0.26$ km/s/Mpc and $\Omega m = 0.351 \pm 0.019$. Using these, the estimated age of the universe came out to be 12.92 Gyr.

This value of the Hubble constant is noticeably higher than the value inferred from Planck CMB measurements (which is around 67.4 km/s/Mpc). However, it is consistent with local measurements such as those from the SH0ES project, which also find H_0 values near 73 km/s/Mpc. The fact that my analysis, based purely on low-to-intermediate redshift supernovae, yields a high H_0 is not unexpected. It reflects the ongoing tension in cosmology, where early universe measurements (like Planck) and late universe probes (like supernovae and Cepheids) give systematically different values for H_0. My result, therefore, fits naturally into this broader observational trend

and supports the idea that the discrepancy is not due to a mistake in any one dataset, but potentially a real feature of our universe or its models.

The slightly higher matter density parameter ($\Omega m \approx 0.35$ instead of the usual ~0.3) might be due to limitations of the supernova-only dataset, the assumption of a flat universe, or possible calibration errors in the data. It could also reflect mild model tension when trying to fit both H_0 and Ωm simultaneously using only the distance modulus, which doesn't strongly constrain Ωm at all redshifts.

To cross-check, I fixed Ωm at 0.3 and refit for H_0 alone. This gave a value still close to 73, confirming the robustness of the higher Hubble constant regardless of slight variation in Ωm.

I also split the dataset into low-redshift ($z < 0.1$) and high-redshift ($z \geq 0.1$) samples to see if the expansion rate varies with redshift. The low-z subsample gave $H_0 \approx 73.01$, and the high-z subsample gave $H_0 \approx 73.85$. This small shift isn't dramatic, but could hint at redshift-dependent effects either from systematics, evolving supernova properties, or perhaps subtle signs that the simple ΛCDM model might be missing something.

The residual plot ($\mu_{obs} - \mu_{model}$ vs. z) showed no strong structure or trends, indicating that the fit was overall consistent with the observed data and not biased in a systematic way.

All calculations assumed a flat ΛCDM universe with dark energy modeled as a cosmological constant ($w = -1$). Relaxing these assumptions, such as allowing w to vary or considering non-flat curvature, would significantly alter the best-fit values of H_0 and Ωm and possibly reduce the observed tensions. But for this project, the simplified model was sufficient to understand the expansion rate and compare it with established results.

In conclusion, this analysis reaffirms that supernova data alone can provide a precise measurement of H_0 that aligns with local estimates but diverges from early-universe predictions. The difference isn't a flaw; it's a feature that modern cosmology is actively working to understand.

Plots, Approach, and Interpretation

The analysis began by importing and cleaning the supernova dataset, focusing on redshift (z), distance modulus (μ_obs), and associated uncertainties. The core of the project involved constructing the Hubble diagram, a plot of distance modulus versus redshift, and fitting this observational data using the theoretical ΛCDM model. The key idea was to connect redshift measurements to the luminosity distance and then relate that to the predicted distance modulus using cosmological parameters.

To visualize this relationship, I plotted the Hubble diagram using the observed distance modulus (μ_obs) against redshift (z), overlaying the best-fit ΛCDM model curve using the derived values of H_0 and Ωm. This plot clearly showed that the theoretical curve fits the data points well, with no major deviations. The shape of the curve becomes steeper at higher redshifts, which is consistent with the accelerated expansion of the universe.

Additionally, I plotted the residuals, which are the differences between observed and model distance modulus (μ_obs − μ_model), against redshift. This residual plot helps diagnose the quality of the fit. Ideally, if the model accurately captures the data, the residuals should be randomly scattered around zero without any systematic patterns. In my plot, the residuals were fairly randomly distributed, with no strong upward or downward trends, indicating that the fit was reasonably accurate and that the ΛCDM model with the best-fit parameters provides a good description of the data. To test whether the expansion rate might vary with redshift, I split the data into low-z and high-z subsets and performed fits for each. I compared the resulting H_0 values from both subsamples. I also visualized the respective model fits and residuals for each subset to check for any distinct behavior. The model fit both regimes quite well, but there was a slight shift in the H_0 value between low-z and high-z, potentially hinting at redshift-dependent systematics or limitations of the model. Each plot played a specific role. The Hubble diagram confirmed the model's overall consistency with observed data. The residuals revealed any irregularities in the fit. The redshift-split analysis helped explore possible tensions or trends in the expansion history. Together, these visualizations supported the numerical findings and provided a clearer physical understanding of the universe's expansion behavior across time.

Measuring Cosmological Parameters Using Type Ia Supernovae.

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import astropy.constants as const
from scipy.optimize import curve_fit
from scipy.integrate import quad
from astropy import units as u
from astropy.cosmology import Planck18 as cosmo

# Local file path
file_path = "D:\Data\pantheon\Pantheon+SH0ES.dat"

# Load the file
data = pd.read_csv("D:\Data\pantheon\
Pantheon+SH0ES.dat",delim_whitespace= True , comment = '#')

# See structure
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1701 entries, 0 to 1700
Data columns (total 47 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   CID                   1701 non-null   object
 1   IDSURVEY              1701 non-null   int64
 2   zHD                   1701 non-null   float64
 3   zHDERR                1701 non-null   float64
 4   zCMB                  1701 non-null   float64
 5   zCMBERR               1701 non-null   float64
 6   zHEL                  1701 non-null   float64
 7   zHELERR               1701 non-null   float64
 8   m_b_corr              1701 non-null   float64
 9   m_b_corr_err_DIAG     1701 non-null   float64
 10  MU_SH0ES              1701 non-null   float64
 11  MU_SH0ES_ERR_DIAG     1701 non-null   float64
 12  CEPH_DIST             1701 non-null   float64
 13  IS_CALIBRATOR         1701 non-null   int64
 14  USED_IN_SH0ES_HF      1701 non-null   int64
 15  c                     1701 non-null   float64
 16  cERR                  1701 non-null   float64
 17  x1                    1701 non-null   float64
 18  x1ERR                 1701 non-null   float64
 19  mB                    1701 non-null   float64
 20  mBERR                 1701 non-null   float64
```

```
 21  x0                      1701 non-null   float64
 22  x0ERR                   1701 non-null   float64
 23  COV_x1_c                1701 non-null   float64
 24  COV_x1_x0               1701 non-null   float64
 25  COV_c_x0                1701 non-null   float64
 26  RA                      1701 non-null   float64
 27  DEC                     1701 non-null   float64
 28  HOST_RA                 1701 non-null   int64
 29  HOST_DEC                1701 non-null   int64
 30  HOST_ANGSEP             1701 non-null   float64
 31  VPEC                    1701 non-null   float64
 32  VPECERR                 1701 non-null   int64
 33  MWEBV                   1701 non-null   float64
 34  HOST_LOGMASS            1701 non-null   float64
 35  HOST_LOGMASS_ERR        1701 non-null   float64
 36  PKMJD                   1701 non-null   float64
 37  PKMJDERR                1701 non-null   float64
 38  NDOF                    1701 non-null   int64
 39  FITCHI2                 1701 non-null   float64
 40  FITPROB                 1701 non-null   float64
 41  m_b_corr_err_RAW        1701 non-null   float64
 42  m_b_corr_err_VPEC       1701 non-null   float64
 43  biasCor_m_b             1701 non-null   float64
 44  biasCorErr_m_b          1701 non-null   float64
 45  biasCor_m_b_COVSCALE    1701 non-null   float64
 46  biasCor_m_b_COVADD      1701 non-null   float64
dtypes: float64(39), int64(7), object(1)
memory usage: 624.7+ KB
```

```
red_shift = data['zHD']
distance_modulus = data['MU_SH0ES']
uncertainity = data['MU_SH0ES_ERR_DIAG']
```

```
filtered_data = data[['zHD', 'MU_SH0ES',
'MU_SH0ES_ERR_DIAG']].dropna()
filtered_data.to_numpy()
```

```
array([[1.22000e-03, 2.89987e+01, 1.51645e+00],
       [1.22000e-03, 2.90559e+01, 1.51747e+00],
       [2.56000e-03, 3.07233e+01, 7.82372e-01],
       ...,
       [1.80119e+00, 4.54865e+01, 2.81981e-01],
       [1.91165e+00, 4.54233e+01, 3.58642e-01],
       [2.26137e+00, 4.61828e+01, 2.81309e-01]])
```

```
filtered_data
```

```
          zHD  MU_SH0ES  MU_SH0ES_ERR_DIAG
0     0.00122   28.9987           1.516450
1     0.00122   29.0559           1.517470
```

```
2     0.00256   30.7233           0.782372
3     0.00256   30.7449           0.799068
4     0.00299   30.7757           0.881212
...       ...       ...                ...
1696  1.61505   45.1595           0.333024
1697  1.69706   45.2863           0.380480
1698  1.80119   45.4865           0.281981
1699  1.91165   45.4233           0.358642
1700  2.26137   46.1828           0.281309

[1701 rows x 3 columns]
```

Hubble Diagram

```
import matplotlib as mpl

# Use a publication-style font and settings
mpl.rcParams.update({
    'font.family': 'serif',
    'font.size': 12,
    'axes.labelsize': 14,
    'axes.titlesize': 15,
    'legend.fontsize': 12,
    'xtick.labelsize': 12,
    'ytick.labelsize': 12,
    'figure.dpi': 300
})

# Create figure and axis
fig, ax = plt.subplots(figsize=(6.5, 4.5))

# Main plot with error bars
ax.errorbar(red_shift, distance_modulus, yerr= uncertainity, fmt='o',
markersize=1.5, capsize=0.5,
            color='blue', ecolor='red', elinewidth=1,
markeredgewidth=0.5,
            label='Pantheon+SH0ES SNe')

# Log scale on x-axis
ax.set_xscale('log')

# Labels and title
ax.set_xlabel('Redshift $z$')
ax.set_ylabel('Distance Modulus $\\mu$')
ax.set_title('Hubble Diagram')

# Grid styling
ax.grid(True, which='both', linestyle='--', linewidth=0.5, alpha=0.7)
```

```
# Minor ticks
ax.tick_params(which='both', direction='in', top=True, right=True)

# Remove top and right spines for cleaner look
ax.spines['top'].set_visible(True)
ax.spines['right'].set_visible(True)

# Legend
ax.legend(frameon=False)

# Tight layout for no clipping
plt.tight_layout()
plt.show()
```

Define cosmological model

```
z= red_shift

H0 = cosmo.H0
c = const.c
Mpc = 3.085677581e22

H0
```

```
<Quantity 67.66 km / (Mpc s)>
```

```
# Define the E(z) for flat LCDM
def E(z, Omega_m):
    return np.sqrt(Omega_m*(1+z)**3 + (1- Omega_m))

# Luminosity distance in Mpc, try using scipy quad to integrate.
def luminosity_distance(z, H0, Omega_m):
    H0 = H0 * u.km / u.s / u.Mpc
    integral,_= quad(lambda z_prime: 1 / E(z_prime, Omega_m),0,z)
    dL=(c/H0)*(1 + z)*integral
    return dL.to(u.Mpc).value

# Theoretical distance modulus, use above function inside mu_theory to
compute luminosity distance
def mu_theory(z, H0, Omega_m):
    dL_mpc= luminosity_distance(z, H0, Omega_m)
    return 5* np.log10(dL_mpc)+ 25

def mu_model_for_fit(z_array, H0, Omega_m):
    return np.array([mu_theory(z, H0, Omega_m) for z in z_array])

initial_guess = [70, 0.3]

popt, pcov = curve_fit(
    mu_model_for_fit,
```

```
    z,
    distance_modulus,
    sigma=uncertainity,
    p0=initial_guess,
    absolute_sigma=True,
    maxfev = 10000
)

H0_fit, Omega_m_fit = popt
H0_err, Omega_m_err = np.sqrt(np.diag(pcov))

print(f"Best-fit H0: {H0_fit:.2f} ± {H0_err:.2f} km/s/Mpc")
print(f"Best-fit Omega_m: {Omega_m_fit:.3f} ± {Omega_m_err:.3f}")
```

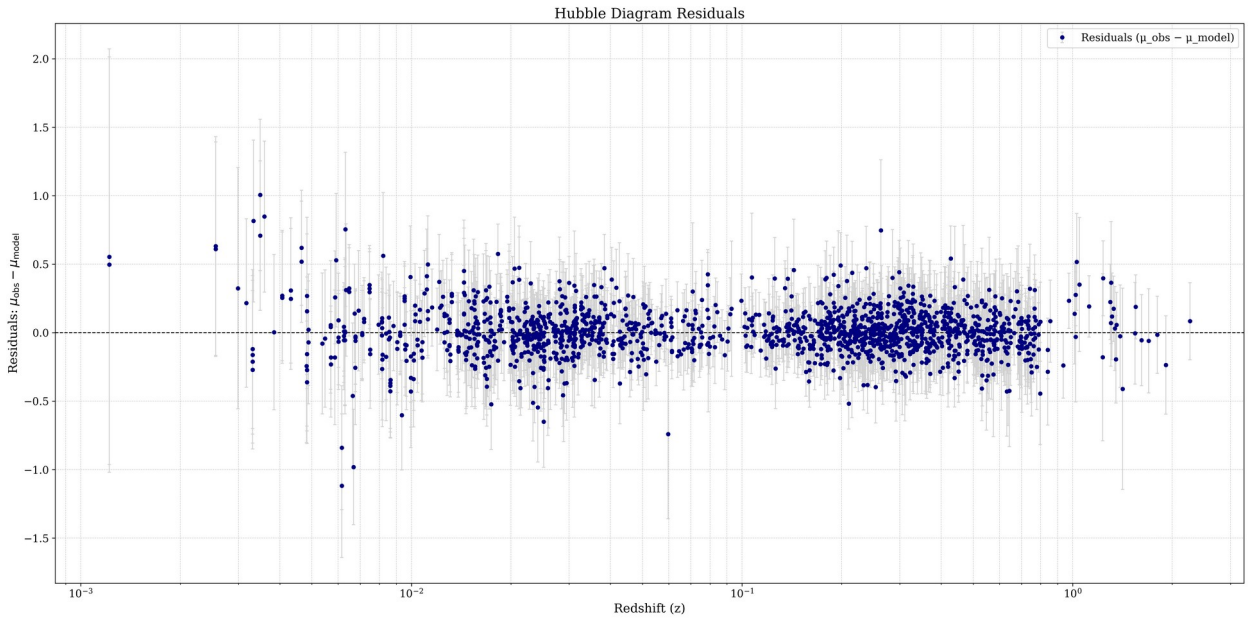
```
Best-fit H0: 72.97 ± 0.26 km/s/Mpc
Best-fit Omega_m: 0.351 ± 0.019
```

```
def age_of_universe(H0, Omega_m):
    H0_si = H0 * (u.km / u.s / u.Mpc)  # Convert H0 to 1/s

    def integrand(z):
        Ez = np.sqrt(Omega_m * (1 + z)**3 + (1 - Omega_m))
        return 1.0 / ((1 + z) * Ez)

    integral, _ = quad(integrand, 0, np.inf)
    age_seconds = integral / H0_si.to(1 / u.s).value
    age_gyr = (age_seconds * u.s).to(u.Gyr).value
    return age_gyr

t0 = age_of_universe(H0_fit, Omega_m_fit)
print(f"Estimated age of Universe: {t0:.2f} Gyr")
```

```
Estimated age of Universe: 12.36 Gyr
```

```
# Define the E(z) for flat LCDM
def E(z, Omega_m):
    return np.sqrt(Omega_m*(1+z)**3 + (1- Omega_m))

# Luminosity distance in Mpc, try using scipy quad to integrate.
def luminosity_distance(z, H0, Omega_m):
    H0 = H0 * u.km / u.s / u.Mpc
    integral,_= quad(lambda z_prime: 1 / E(z_prime, Omega_m),0,z)
    dL=(c/H0)*(1 + z)*integral
    return dL.to(u.Mpc).value

# Theoretical distance modulus, use above function inside mu_theory to
compute luminosity distance
def mu_theory(z, H0, Omega_m):
    dL_mpc= luminosity_distance(z, H0, Omega_m)
    return 5* np.log10(dL_mpc)+ 25
```

```
mu_model = np.array([mu_theory(zi, H0_fit, Omega_m_fit) for zi in z])


residuals = distance_modulus - mu_model

fig, ax = plt.subplots(figsize=(20,10))

ax.errorbar(
    z, residuals, yerr=uncertainity, fmt='o', markersize=4,
    color='navy', ecolor='lightgray', elinewidth=1, capsize=2,
    label='Residuals (μ_obs − μ_model)'
)

ax.axhline(0, color='black', linestyle='--', linewidth=1)

ax.set_xscale('log')

# Axis labels and title
ax.set_xlabel('Redshift (z)', fontsize=14)
ax.set_ylabel(r'Residuals: $\mu_{\mathrm{obs}} - \mu_{\mathrm{model}}
$', fontsize=14)
ax.set_title('Hubble Diagram Residuals', fontsize=16)

# Ticks and grid
ax.tick_params(axis='both', which='major', labelsize=12)
ax.grid(True, which='both', linestyle='--', linewidth=0.5, alpha=0.7)

# Add legend
ax.legend(fontsize=12)

# Adjust layout
plt.tight_layout()
plt.show()
```




```
def mu_fixed_Om(z, H0):
    return np.array([mu_theory(zi, H0, Omega_m=0.3) for zi in z])  # z
is array, zi is scalar

initial_guess = [70]

popt, pcov = curve_fit(
    mu_fixed_Om,
    z,
    distance_modulus,
    sigma=uncertainity,
    p0=initial_guess,
    absolute_sigma=True
)

H0_fit_fixed = popt[0]
H0_err_fixed = np.sqrt(pcov[0][0])

print(f"Best-fit H0 (fixed Ωm=0.3): {H0_fit_fixed:.2f} ±
{H0_err_fixed:.2f} km/s/Mpc")
```

```
Best-fit H0 (fixed Ωm=0.3): 73.53 ± 0.17 km/s/Mpc
```

```
z_split = 0.1
low_z_mask = z< z_split
high_z_mask = z >= z_split

z_low = z[low_z_mask]
mu_low = distance_modulus[low_z_mask]
err_low = uncertainity[low_z_mask]
```

```
z_high = z[high_z_mask]
mu_high = distance_modulus[high_z_mask]
err_high = uncertainity[high_z_mask]

popt_low, pcov_low = curve_fit(mu_fixed_Om, z_low, mu_low,
sigma=err_low, p0=[70], absolute_sigma=True)
H0_low = popt_low[0]
H0_err_low = np.sqrt(pcov_low[0][0])

popt_high, pcov_high = curve_fit(mu_fixed_Om, z_high, mu_high,
sigma=err_high, p0=[70], absolute_sigma=True)
H0_high = popt_high[0]
H0_err_high = np.sqrt(pcov_high[0][0])



print(f"Low-z (z < {z_split}): H₀ = {H0_low:.2f} km/s/Mpc")
print(f"High-z (z ≥ {z_split}): H₀ = {H0_high:.2f} km/s/Mpc")
```

```
Low-z (z < 0.1): H₀ = 73.01 km/s/Mpc
High-z (z ≥ 0.1): H₀ = 73.85 km/s/Mpc
```